Dot Product Performance Benchmark Report

1 实验概述

1.1 实验目标

对比 C 语言和 Java 在计算向量点积时的性能差异，覆盖多种数据类型和向量规模。

1.2 实验环境

- **平台**：macOS (Apple Silicon)
- **C 编译器**：gcc（两种配置：无优化 `-O0`，最高优化 `-O3`）
- **Java**：JDK（默认 JIT 编译）
- **随机种子**：42（固定，确保可复现）

1.3 实验设计

- **数据类型**：float (4B), double (8B), int (4B), short (2B), signed char / byte (1B)
- **向量规模** n：128, 1000, 10^4, 10^5, 10^6, 10^7, 10^8
- **计时方式**：C 使用 `<time.h>`，Java 使用 `System.nanoTime()`

2 性能数据

2.1 按数据类型分组

float (4 bytes)

n	C-O0 (ms)	C-O3 (ms)	Java (ms)
128	0.000977	0.000000	0.001541
1,000	0.006104	0.000977	0.006875
10,000	0.063965	0.010010	0.055584
100,000	0.506104	0.092041	0.684375
1,000,000	1.857178	0.587158	0.564042
10,000,000	13.484863	5.407959	5.324083
100,000,000	133.640137	77.952881	53.745917

double (8 bytes)

n	C-O0 (ms)	C-O3 (ms)	Java (ms)
128	0.000000	0.000000	0.002042
1,000	0.004150	0.000000	0.005792
10,000	0.030029	0.005127	0.051625
100,000	0.079834	0.049072	0.563750
1,000,000	1.458984	0.862793	0.558000
10,000,000	18.525146	11.900146	5.368208
100,000,000	188.330078	204.753906	57.911875

int (4 bytes)

n	C-O0 (ms)	C-O3 (ms)	Java (ms)
128	0.000000	0.000000	0.001625
1,000	0.001221	0.000000	0.006375
10,000	0.010010	0.000977	0.054166
100,000	0.076904	0.008057	0.583958
1,000,000	0.788086	0.096191	0.454959
10,000,000	7.887207	0.962158	2.344083
100,000,000	81.145996	9.700928	23.592250

short (2 bytes)

n	C-O0 (ms)	C-O3 (ms)	Java (ms)
128	0.000000	0.000000	0.001708
1,000	0.000977	0.000000	0.006584
10,000	0.008057	0.000977	0.058542
100,000	0.078125	0.016846	0.596709
1,000,000	0.778076	0.173828	0.290000
10,000,000	7.851074	3.496094	2.330875
100,000,000	79.534912	16.736084	23.499458

signed char / byte (1 byte)

n	C-O0 (ms)	C-O3 (ms)	Java (ms)
128	0.000000	0.000977	0.001792
1,000	0.000977	0.000000	0.006250
10,000	0.008057	0.001953	0.055708
100,000	0.079834	0.017090	0.590333
1,000,000	0.770996	0.184082	0.249167
10,000,000	7.874023	1.681885	2.342375
100,000,000	101.544922	24.444092	23.563875

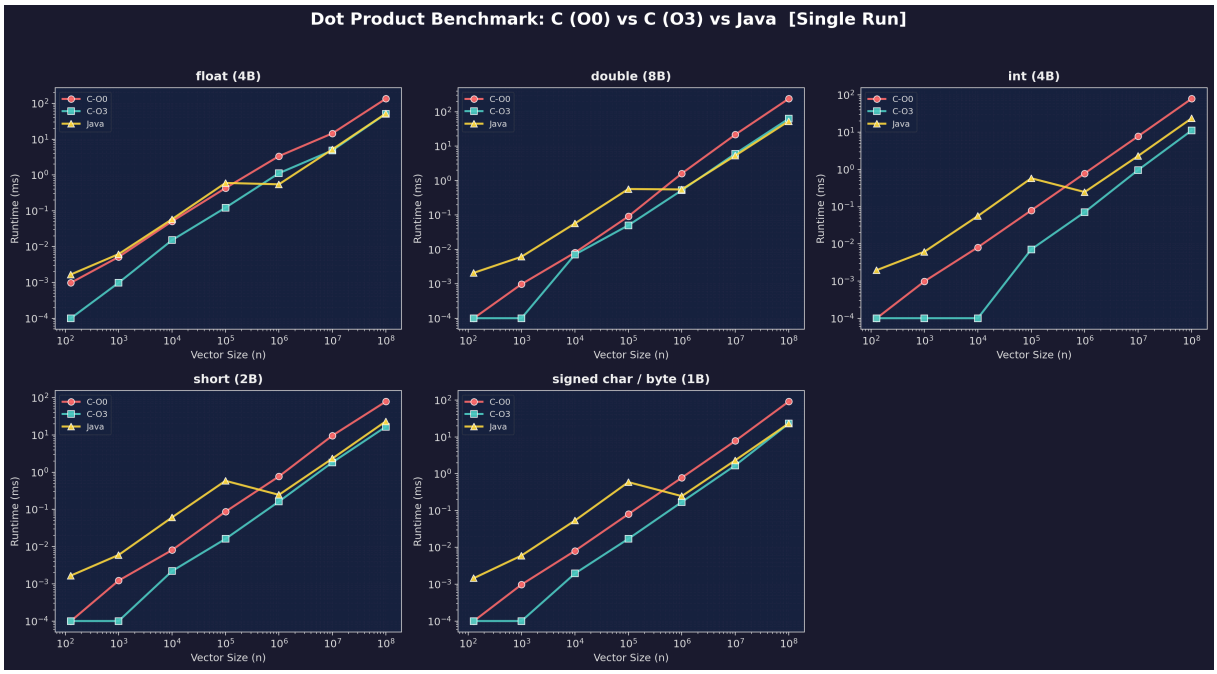

图 1: 各数据类型在不同向量规模下的运行时间对比（对数坐标）

2.2 加速比（$n = 10^8$）

数据类型	C-O3 vs C-O0	Java vs C-O0	Java vs C-O3
float	1.71x	2.49x (Java faster)	1.45x (Java faster)
double	0.92x	3.25x (Java faster)	3.54x (Java faster)
int	8.36x	3.44x (Java faster)	0.41x (C-O3 faster, 2.43x)
short	4.75x	3.38x (Java faster)	0.71x (C-O3 faster, 1.40x)
signed char	4.16x	4.31x (Java faster)	1.04x (roughly equal)

3 分析

3.1 C-O0 vs C-O3：编译器优化的效果

`gcc -O3` 在编译时对代码施加了一系列静态优化，效果在不同数据类型上差异显著：

数据类型	O3/O0 加速比	分析
int	**8.36x**	整数运算最受益：O3 可将 int 乘加循环充分向量化（NEON 128-bit 寄存器一次处理 4 个 int），且无浮点精度约束
short	4.75x	向量化收益更高（128/16=8 个/次），但 short→int 的类型提升增加了额外指令
signed char	4.16x	理论并行度最高（128/8=16 个/次），但类型提升开销更大
float	1.71x	浮点向量化受限于 IEEE 754 精度要求，编译器不能随意重排加法顺序
double	**0.92x**	O3 反而略慢于 O0，可能是 O3 的某些变换（如指令调度）增加了 cache miss 或破坏了流水线效率

关键发现：O3 对整数类型的加速最为显著（int 8.36x），这符合 SIMD 向量化的理论预期——元素越小并行度越高，且整数运算没有浮点精度约束。但 double 出现了意外的 O3 劣化现象。

3.2 Java 的 JIT（Just-In-Time）编译现象

3.2.1 现象观察

Java 的运行时间出现了一个**反直觉**的现象：

数据类型	$n = 10^5$	$n = 10^6$	变化
float	0.684375 ms	0.564042 ms	n 增大 10 倍，时间反而**下降** 18%
double	0.563750 ms	0.558000 ms	n 增大 10 倍，时间几乎**不变**
int	0.583958 ms	0.454959 ms	时间下降 **22%**
short	0.596709 ms	0.290000 ms	时间下降 **51%**
byte	0.590333 ms	0.249167 ms	时间下降 **58%**

此外，所有类型在 $n = 10^5$ 时耗时处于相近范围（0.56~0.68ms），与数据类型大小关系不大——暗示此阶段的耗时主要由 JIT 编译的固定开销主导，而非真正的计算。

推测：JVM 的 JIT 编译器在 $n = 10^5$ 到 10^6 之间完成了编译切换。之前的数据点反映的是解释器逐条分派字节码的性能（开销较高），之后反映的是 JIT 编译器生成的专用本机机器码的性能（经过深度优化）。

3.2.2 背景知识：分层编译与 OSR

JVM 采用**分层编译（Tiered Compilation）**策略。需要注意，CPU 最终执行的**始终是机器码**——区别在于执行模式：

```
阶段 0:   解释执行       -> 逐条读取并执行字节码（解释器通过预编写的
    codelet
                          逐条分派，每条字节码都有 dispatch 开销），
                             最慢
阶段 1-3: C1 编译执行    -> C1 编译器快速生成本机代码，基础优化
阶段 4:   C2 编译执行    -> C2 编译器深度优化，生成高性能本机代码
```

OSR（On-Stack Replacement）：JIT 不需要等方法返回才能切换。当循环的回边计数达到阈值时，JVM 在循环头处将当前栈帧原地替换为编译后的机器码版本，后续迭代直接以本机代码运行。

触发阈值：通过 `java -XX:+PrintFlagsFinal -version | grep CompileThreshold` 查得本机（OpenJDK 25）实际配置：

参数	默认值	含义
`Tier3CompileThreshold`	2,000	触发 C1 全 profiling 编译
`Tier4CompileThreshold`	15,000	触发 C2 最高优化编译
`CompileThreshold`	10,000	关闭分层编译时的遗留默认值

注意：分层编译下 JVM 使用 `AdvancedThresholdPolicy`，实际触发时机受编译队列负载等动态因素影响，并非简单的固定次数阈值。

3.2.3 实验 1：JitWarmup 控制实验

设计控制实验 `JitWarmup.java`：固定 $n = 10{,}000$，重复调用 `dotFloat` 30 次，观察是否在 n 不变的情况下仍会出现速度跳变。

```
Round      Time (ms)
-----      ---------
  1         0.1140    <- 解释器分派执行
  2         0.1132
  ...
  7         0.1100
  8         0.0235    <- 速度突变（JIT 编译完成）
  9         0.0229
 ...        ~0.023    <- 此后稳定
```

- Round 1–7：平均 ~0.112 ms（解释器分派执行）
- Round 8–30：平均 ~0.023 ms（JIT 编译后的本机码执行）
- **加速比**：$0.112/0.023 \approx$ **4.9x**

结论：n 始终为 10,000 不变，速度仍在第 7→8 轮突变 ~4.9x。这证明速度跳变**与 n 的大小无关**，而是由累计执行次数驱动的 JIT 编译引起。

3.2.4 实验 2：PrintCompilation 编译事件观察

使用 JVM 诊断参数运行：

```
java -XX:+PrintCompilation Dotproduct 2>&1 | grep "Dotproduct::dot"
```

输出（筛选 `dot` 系列方法）：

```
时间戳    编号    标记   层级   方法                              字节码
------  -----  ---  ---  ----------------------------   ---------
1325    2016    %    3    Dotproduct::dotFloat    @ 4      (28 bytes
   )
1325    2017         3    Dotproduct::dotFloat             (28
   bytes)
3286    2044    %    4    Dotproduct::dotDouble   @ 5      (32 bytes
   )
3316    2046         4    Dotproduct::dotDouble            (32
   bytes)
7000    2109    %    4    Dotproduct::dotInt      @ 5      (34 bytes
   )
7008    2112         4    Dotproduct::dotInt               (34
   bytes)
```

```
8821    2125    %    4    Dotproduct::dotShort   @ 5      (34 bytes
   )
8835    2128         4    Dotproduct::dotShort             (34
   bytes)
10583   2136    %    4    Dotproduct::dotByte    @ 5      (34 bytes
   )
10595   2143         4    Dotproduct::dotByte              (34
   bytes)
```

字段含义：% = OSR 编译（循环中热替换），层级 3 = C1 编译，层级 4 = C2 最高优化，@ N = OSR 入口的字节码偏移。每个方法成对出现：OSR 版（替换当前循环）+ 标准版（供后续调用使用）。

以下对该输出中的三个关键观察进行分析：

观察 1：dotFloat 先经 C1（层级 3），其余方法直接 C2（层级 4）

dotFloat 是第一个被编译的用户方法（时间戳 1325ms），此时出现在层级 3（C1）。而从 dotDouble 起，所有方法直接出现在层级 4（C2）。

证据：查看不加 grep 过滤的完整编译日志，在 dotFloat 被编译的同一时间（~50ms），JVM 正在密集编译大量 JDK 底层方法：

```
时间戳   方法                                          层级
------  ----------------------------------------  ----
13      java.lang.String::hashCode                  3
17      java.util.Random::next                      3
19      java.util.HashMap::putVal                   3
19      java.lang.Object::<init>                    4
48      java.util.regex.Pattern$BmpCharPropertyGreedy  4
54      java.util.Random::nextFloat                 4   <- 此时
   dotFloat 刚好变热
```

C2 编译器速度慢但优化深度高，此时其编译队列被这些底层方法挤占。JVM 的 AdvancedThresholdPolicy 检测到 C2 队列拥堵，选择先用 C1 快速编译 dotFloat 以尽快获得比解释器更好的性能。

到 dotDouble 被调用到热点时（时间戳 3286ms，距首次编译已过 ~2 秒），大量底层方法已编译完成，C2 队列空闲，因此 JVM 直接将其提交到 C2。后续方法同理。

观察 2：OSR 入口偏移 @ 4 vs @ 5

dotFloat 的 OSR 入口在字节码偏移 4，其余方法在偏移 5。使用 javap -c Dotproduct 反编译字节码可以解释原因：

```
// dotFloat: sum 为 float（占 1 个 local slot）
0: fconst_0          // float sum = 0.0f
```

```
1: fstore_2          // -> local[2]（短格式，1 byte）
2: iconst_0          // int i = 0
3: istore_3          // -> local[3]（短格式，1 byte）
4: iload_3           // <- 循环头（OSR 入口 @ 4）

// dotDouble: sum 为 double（占 2 个 local slot）
0: dconst_0          // double sum = 0.0
1: dstore_2          // -> local[2-3]（占 2 slot）
2: iconst_0          // int i = 0
3: istore 4          // -> local[4]（长格式，2 bytes）
5: iload  4          // <- 循环头（OSR 入口 @ 5）
```

float 的 sum 占 1 个 slot，循环变量 i 分配到 local[3]，使用单字节短格式指令 istore_3；double/long 的 sum 占 2 个 slot，i 被推到 local[4]，使用双字节长格式指令 istore 4，导致循环头偏移多 1。

观察 3：字节码长度 28 / 32 / 34 bytes

证据：使用 javap -c Dotproduct 反编译，对比三个方法的循环体关键指令差异：

```
// dotFloat（28 bytes）—— 循环体部分
 4: iload_3           // 1 byte（短格式）
10: fload_2           // 1 byte
12: iload_3           // 1 byte（短格式）
13: faload            // 直接取 float
15: iload_3           // 1 byte（短格式）
19: fstore_2          // 1 byte
20: iinc 3, 1         // 3 bytes

// dotDouble（32 bytes）—— 循环体部分（+4B）
 5: iload  4          // 2 bytes（长格式，+1）
12: dload_2           // 1 byte
14: iload  4          // 2 bytes（长格式，+1）
16: daload            // 直接取 double
18: iload  4          // 2 bytes（长格式，+1）
23: dstore_2          // 1 byte
24: iinc 4, 1         // 3 bytes  -> 注意：iload 4 出现 3 次 +
   istore 4 出现 1 次 = +4B

// dotInt（34 bytes）—— 循环体部分（+2B vs dotDouble）
 5: iload  4          // 同 dotDouble
16: iaload            // 取 int
17: i2l               // <- 额外指令（+1B）
```

```
21: iaload
22: i2l                    // <- 额外指令（+1B）   -> 两条 i2l 共 +2B
```

汇总：

方法	字节码	差异原因
`dotFloat`	28 bytes	`float` 单 slot + 短格式指令，最紧凑
`dotDouble`	32 bytes	`double` 双 slot + 长格式 `istore/iload`（+4B）
`dotInt`	34 bytes	同 double 的 slot 布局（`long` 返回值）， 额外两条 `i2l` 类型转换指令（+2B）
`dotShort`	34 bytes	同 `dotInt`（`saload` + `i2l` ×2）
`dotByte`	34 bytes	同 `dotInt`（`baload` + `i2l` ×2）

3.2.5 实验 3：JIT 优化内容验证

为定量验证 JIT 各层级和 C2 各项优化的实际贡献，设计对照实验：使用不同 JVM 参数运行同一 benchmark（$n = 10^8$，每种配置 3 次取中位值）。

第一组：层级对比

JVM 配置	float	double	int	short	byte
`-Xint`（纯解释器）	503.3	554.0	505.6	509.8	507.2
`-XX:TieredStopAtLevel=3`（C1）	106.1	112.3	124.6	126.2	123.1
默认（C2）	54.3	55.6	26.3	26.0	23.9

表 1: 不同编译层级下的耗时（ms，$n = 10^8$）

加速比	float	double	int	short	byte
解释器 → C1	4.7x	4.9x	4.1x	4.0x	4.1x
C1 → C2	2.0x	2.0x	4.7x	4.9x	5.2x
解释器 → C2	9.3x	10.0x	19.2x	19.6x	21.2x

表 2: 各层级间的加速比

关键发现：

- **解释器 → C1**：所有类型均获得 ~4–5x 加速，且浮点与整数差异不大——C1 的基础优化（简单内联、基本寄存器分配）对所有类型普遍有效。
- **C1 → C2：整数类型额外获得 ~5x 加速**（vs 浮点的 2x）。这说明 C2 的深度优化对整数运算的收益远大于浮点运算。

第二组：C2 单项优化验证

在默认 C2 编译的基础上，逐一关闭特定优化：

关闭的优化	JVM 参数	float	double	int	short	byte
（基准：C2 默认）	—	54.3	55.6	26.3	26.0	23.9
SIMD 向量化	`-XX:-UseSuperWord`	53.3	52.7	26.8	25.3	23.8
循环展开	`-XX:LoopUnrollLimit=0`	62.5	64.2	33.5	33.9	34.2
循环谓词	`-XX:-UseLoopPredicate`	52.7	57.4	25.2	24.0	23.8

表 3: 关闭单项 C2 优化后的耗时（ms，$n = 10^8$）

分析：

- **循环展开**（`LoopUnrollLimit=0`）：**唯一有显著影响的优化**。关闭后，浮点类型慢 ∼15%（54→62ms），整数类型慢 ∼28–43%（26→34ms）。这证实 C2 确实应用了循环展开优化。
- **SIMD 向量化**（`UseSuperWord`）：关闭后耗时**几乎无变化**。这表明 HotSpot 的 SuperWord 自动向量化在本 benchmark 的点积循环（包含累加归约模式）中并未生效，C2 的性能优势并非来自 SIMD。
- **循环谓词**（`UseLoopPredicate`）：关闭后耗时**几乎无变化**。可能是因为 C2 通过其他机制（如 range check elimination）已经消除了数组边界检查。

综合结论：C2 相对于 C1 的性能优势主要来自**循环展开**（已验证）和**更优的整体代码生成**（更好的寄存器分配、指令调度等，这些无法通过单个开关关闭来验证，但从 C1→C2 的巨大加速比可以推断）。原本假设的 SIMD 向量化在本 benchmark 中并未贡献性能提升。

3.3 数据类型间的差异

在 $n = 10^8$（JIT 已充分生效）时，不同数据类型的性能特征呈现清晰规律：

整数类型 vs 浮点类型（C-O3）

类型	C-O3 耗时 (ms)	相对 int 的倍数
int	9.700928	1.0x
short	16.736084	1.7x
signed char	24.444092	2.5x
float	77.952881	8.0x
double	204.753906	21.1x

整数运算大幅快于浮点运算。虽然 short/char 的理论 SIMD 并行度更高（8 个/16 个 vs int 的 4 个），但实际性能反而不如 int，原因是 C 语言的**整数提升（integer promotion）规则**：short/char 在参与运算前必须先被提升为 int，增加了额外的类型转换指令。

浮点类型 double 为什么最慢?

double（8 字节）在 $n = 10^8$ 时 C-O3 耗时 204.75ms，是 float（77.95ms）的 2.6 倍。原因：

1. **SIMD 并行度减半**：128-bit 寄存器只能装 2 个 double vs 4 个 float
2. **内存带宽翻倍**：加载 10^8 个 double 需要 1.6GB 数据 vs float 的 0.8GB，受限于内存带宽

Java 在浮点和整数上的不同表现

在 $n = 10^8$ 时：

- **float/double**：Java 快于 C-O3（float 1.45x / double 3.54x），JIT 的运行时优化更激进
- **int**：C-O3 反而快于 Java（2.43x），说明 gcc 的静态整数向量化优化更成熟
- **short**：C-O3 略快于 Java（1.40x）
- **byte**：Java 与 C-O3 基本持平

3.4 C-O0 vs Java 的对比意义

在大 n 时，Java **全面快于** C-O0（2.5x–4.3x）。这说明：

- **JIT 编译后的 Java 并不" 慢"**——它慢的只是 warmup 阶段
- 未优化的 C 代码（`-O0`）性能很差，甚至不如带 JIT 的 Java
- 这提示我们：语言选择本身不是性能瓶颈，**编译器优化才是关键**

4 总结

特性	C (`gcc -O3`)	Java (JIT C2)
优化时机	编译时（静态）	运行时（动态）
首次运行	从第一次就快	需要 warmup
SIMD	编译器自动向量化	C2 自动向量化
Profile-Guided	需手动 PGO	自动 profiling
边界检查	无（程序员负责）	自动消除

核心结论:

1. **C 并不总是比 Java 快**。在大规模浮点计算中，JIT 编译后的 Java 甚至超过 C-O3
2. **编译器优化（-O3）的效果远超语言选择**。C-O0 vs C-O3 的差距可达 8.4x（int）
3. **JIT 的 warmup 是 Java 小 n 下慢的主因**，而非语言本身的效率问题
4. **数据类型选择影响显著**：整数远快于浮点，小数据类型受制于类型提升